

represent elements of an array and are identified by their position in array. An array consists of a contiguous region of storage exactly large enough to hold all of its elements.

4.2.3 Array declaration

Array declaration is similar to variable declaration, with the brackets added after identifier:

```
typearray_name[constant-expression]
```

This declares an array named as `array_name` composed of elements of type. The type can be scalar type (except void), user-defined type, pointer, enumeration, or another array. Result of the constant-expression within the brackets determines the number of elements in array. If an expression is given in an array declarator, it must evaluate to a positive constant integer. The value is the number of elements in the array.

Each of the elements of an array is numbered from 0 through the number of elements minus one. If the number is `n`, elements of array can be approached as variables `array_name[0] .. array_name[n-1]` of type.

Here are a few examples of array declaration:

```
#define MAX = 50
intvector_one[10]; /* declares an array of 10 integers */
float  vector_two[MAX];          /* declares an array
of 50 floats */
float  vector_three[MAX - 20]; /* declares an array of
30 floats */
```

Array initialisation

Array can be initialised in declaration by assigning it a comma-delimited sequence of values within braces. When initializing an array in declaration, you can omit the number of elements – it will be automatically determined according to the number of elements assigned. For example:

```
/* Declare an array which holds number of days in each
month: */
int days[12] = {31,28,31,30,31,30,31,31,30,31,30,31};
/* This declaration is identical to the previous one */
int days[] = {31,28,31,30,31,30,31,31,30,31,30,31};
```

If you specify both the length and starting values, the number of starting values must not exceed the specified length. Vice versa is possible, when the